

Skip List and HNSW

Big Data Management and Governance

Prof. Giovanni Simonini, Giovanni Malaguti

University of Modena and Reggio Emilia

September 30, 2026

Details of the Lab

1. SkipList simple implementation;
2. Brief overview of a simple HNSW implementation and layers visualization;
3. Parameters exploration with real-world HNSW and larger testing dataset;
4. Pre/In-search/Post-filtering approaches for HNSW

The code for the lab and other materials is at:

<https://github.com/Stravanni/scalaDM>

Skip Lists

SkipList: Definition

- Linked List + Binary-search tree/sorted arrays
- Each node has 1+ layers, and each layer might be connected to different nodes
- Layers are assigned following a geometric distribution
- We'll use a simple implementation of Skip Lists, but you can extend it to include key-value pairs inside nodes

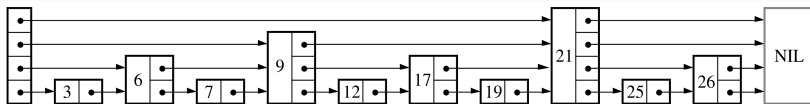


Figure 1: Example of Skip List (from [3])

Algorithm 1 Search

Require: Skip list S with header $head$, value to search q

Ensure: Node with q if present, NIL otherwise

```
1:  $current \leftarrow head$ 
2:  $level \leftarrow \text{max level of the skip list}$ 
3: while  $level \geq 0$  and  $current.pointers[0] \neq NIL$  do
4:    $next \leftarrow current.pointers[level]$ 
5:   if  $next = NIL \vee next.value > q$  then
6:      $level \leftarrow level - 1$ 
7:     continue
8:   end if
9:   if  $next.value = q$  then
10:    return  $next$ 
11:  end if
12:  if  $next.value < q$  then
13:     $current \leftarrow current.pointers[level]$ 
14:  end if
15: end while
16: return  $NIL$ 
```

Random Level Generation

Algorithm 2 Random Level Generation

Require: Probability p , max level $maxLevel$

Ensure: Random level for the new node

- 1: $level \leftarrow 0$
 - 2: **while** $random() < p$ **and** $level < maxLevel$ **do**
 - 3: $level \leftarrow level + 1$
 - 4: **end while**
 - 5: **return** $level$
-

Algorithm 3 Insert

Require: Skip list S with header $head$, value q

Ensure: Skip list updated with the new node

```
1:  $current \leftarrow head$ 
2:  $update \leftarrow$  array of pointers with size  $maxLevel + 1$ 
3:  $level \leftarrow$  current max level
4: for  $i = level$  to 0 do
5:   while  $current.pointers[i] \neq NIL$  and  $current.pointers[i].value < q$  do
6:      $current \leftarrow current.pointers[i]$ 
7:   end while
8:    $update[i] \leftarrow current$ 
9: end for
10:  $current \leftarrow current.pointers[0]$ 
11: if  $current = NIL$  or  $current.value \neq q$  then
12:    $nodeLevel \leftarrow \max(\text{RandomLevel}(), level)$ 
13:    $newNode \leftarrow \text{createNode}(q, nodeLevel)$ 
14:   for  $i = level$  to 0 do
15:      $node \leftarrow update[i]$ 
16:      $newNode.pointers[i] \leftarrow node.pointers[i]$ 
17:      $node.pointers[i] \leftarrow newNode$ 
18:   end for
19: end if
```

HNSW

From Skip Lists to HNSW

- HNSW[1] is a generalization of Skip List data structure
- Each node can have ≥ 1 neighbors at each level, not just one
- The neighbors are a proximity graph of the node

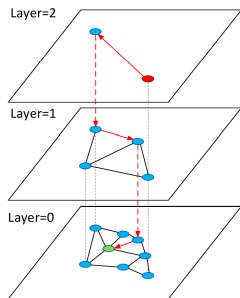


Figure 2: An HNSW graph

Algorithm 4 Search

Require: multilayer graph $hnws$, query element q , number of nearest neighbors to return K , size of the dynamic candidate list $efSearch$

Ensure: K nearest neighbors to q

- 1: $W \leftarrow \emptyset$
 - 2: $ep \leftarrow$ enter point for $hnsw$
 - 3: $L \leftarrow$ level of ep
 - 4: **for** $l_c \leftarrow L \dots 0$ **do**
 - 5: $W \leftarrow \text{SearchLayer}(q, ep, ef = 1, l_c)$
 - 6: $ep \leftarrow$ get nearest element from W to q
 - 7: **end for**
 - 8: $W \leftarrow \text{SearchLayer}(q, ep, ef = efSearch, l_c = 0)$
 - 9: **return** $\text{SelectNeighborsSimple}(q, W, K)$
-

Search: how to search candidates on each layer?

Algorithm 5 SearchLayer

Require: query element q , enter points ep , number of nearest neighbors to q to return ef , layer number l_c

Ensure: ef nearest neighbors to q

```
1:  $v \leftarrow ep$  // visited nodes. Note that the passed  $ep$  is a set of points, not a single node!
2:  $W \leftarrow ep$  // set of final results (updated during search)
3:  $C \leftarrow ep$  // set of candidate nodes for search steps
4: while  $|C| > 0$  do
5:    $c \leftarrow$  extract nearest element from  $C$  to  $q$ 
6:    $f \leftarrow$  get furthest element from  $W$  to  $q$ 
7:   if  $distance(q, c) > distance(q, f)$  then
8:     break
9:   end if
10:  for each  $e \in neighbourhood(c)$  at layer  $l_c$  do
11:    if  $e \notin v$  then
12:       $v \leftarrow v \cup e$ 
13:       $f \leftarrow$  get furthest element from  $W$  to  $q$ 
14:      if  $distance(q, e) < distance(q, f)$  or  $|W| < ef$  then
15:         $C \leftarrow C \cup e$ 
16:         $W \leftarrow W \cup e$ 
17:        if  $|W| > ef$  then
18:          remove furthest element from  $W$  to  $q$ 
19:        end if
20:      end if
21:    end if
22:  end for
23: end while
24: return  $W$ 
```

Search: how to select neighbors (simple version)?

- This is the simplest version of a selection method for neighbors
- It may lead disconnected graphs, where few nodes are not properly linked to the others
- It's simpler and faster than other heuristics (perfect for us)

Algorithm 6 Select Neighbors Simple

Require: base element q , candidate elements C , number of neighbors to return K

Ensure: K nearest elements to q

1: **return** K nearest elements from C to q

Algorithm 7 Insert

Require: multilayer graph $hns w$, new element q , number of connections per node M_{max} , size of dynamic candidate list $efConstruction$, normalization factor m_L

Ensure: update $hns w$ inserting new element q

```
1:  $W \leftarrow \emptyset$  // list for the currently found nearest neighbors
2:  $ep \leftarrow$  get enter point for  $hns w$ 
3:  $L \leftarrow$  level of  $ep$  // current top-layer for  $hns w$ 
4:  $l \leftarrow \text{RandomLevel}(m_L)$ 
5: for  $l_c \leftarrow L \dots l + 1$  do
6:    $W \leftarrow \text{SearchLayer}(q, ep, ef = 1, l_c)$ 
7:    $ep \leftarrow$  get the nearest element from  $W$  to  $q$ 
8: end for
9: for  $l_c \leftarrow \min(L, l) \dots 0$  do
10:   $W \leftarrow \text{SearchLayer}(q, ep, efConstruction, l_c)$ 
11:   $neighbors \leftarrow \text{SelectNeighborsSimple}(q, W, M)$ 
12:  add bidirectional connections from  $neighbors$  to  $q$  at layer  $l_c$ 
13:  for each  $e \in neighbors$  do
14:     $eConn \leftarrow \text{neighbourhood}(e)$  at layer  $l_c$ 
15:    if  $|eConn| \geq M_{max}$  then
16:       $eNewConn \leftarrow \text{SelectNeighborsSimple}(e, eConn, M_{max})$ 
17:      set neighbourhood( $e$ ) at layer  $l_c$  to  $eNewConn$ 
18:    end if
19:  end for
20:   $ep \leftarrow W$ 
21: end for
22: if  $l > L$  then
23:   set enter point for  $hns w$  to  $q$ 
24: end if
```

HNSW and Filters

- HNSW can be used in combination with vectors *metadata*, when similarity alone isn't enough for the user requirements.
- Say that we have a movies dataset: we want movies similar to our query one, but we might want only a specific genre, like "Comedy" or "Drama".
- The distribution of the metadata can affect the results.

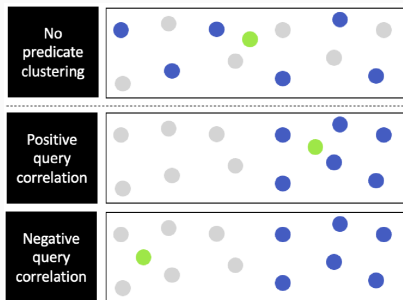


Figure 3: Figure from [2], showing different types of predicate clusterings with a query (green point) and its relevant elements (blue points).

References

-  MALKOV, Y. A., AND YASHUNIN, D. A.
Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2018.
-  PATEL, L., KRAFT, P., GUESTRIN, C., AND ZAHARIA, M.
Acorn: Performant and predicate-agnostic search over vector embeddings and structured data.
Proc. ACM Manag. Data 2, 3 (May 2024).
-  PUGH, W.
Skip lists: a probabilistic alternative to balanced trees.
Commun. ACM 33, 6 (June 1990), 668–676.